

Programação de Sistemas Linux

Aula 02 — Ferramentas de Programação (Parte 1)

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Objetivos do encontro

Ao final deste encontro, você será capaz de:

- ✓ Compilar programas em C com o **GCC**, entendendo o que cada etapa faz
- ✓ Usar flags de **warning** e **otimização** de forma consciente
- ✓ Automatizar a compilação de projetos com **Make** e **Makefile**
- ✓ Encontrar bugs sem usar `printf` por toda parte, com o **GDB**

Regra de hoje

Teoria curta, mão na massa longa. A atividade prática vale mais que os slides.

Notes

Notes

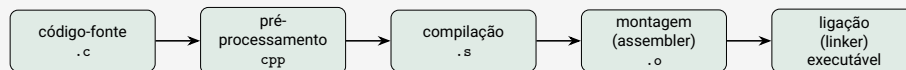


GCC: compilando programas C

Notes

O pipeline de compilação

Compilar não é um passo só — são quatro:



- **Pré-processamento:** expande `#include`, `#define`, macros
- **Compilação:** gera código assembly a partir do C
- **Montagem:** assembly → código de máquina (objeto `.o`)
- **Ligação:** junta os `.o` e bibliotecas em um executável

Notes



Forma mais simples: fazer tudo de uma vez.

```
1 $ gcc programa.c -o programa
2 $ ./programa
```

- gcc chama automaticamente o pré-processador, compilador, montador e o *linker*
- -o programa define o nome do executável (senão, sai a.out)

Pergunta para a turma

O que acontece se você rodar `gcc programa.c` sem `-o`?



Notes

Compilando etapa por etapa

O GCC permite parar em qualquer fase do pipeline:

```
1 $ gcc -E programa.c -o programa.i # so pre-processamento
2 $ gcc -S programa.c -o programa.s # gera assembly (.s)
3 $ gcc -c programa.c -o programa.o # gera objeto (.o), sem linkar
4 $ gcc programa.o -o programa # so a ligacao (linker)
```

- Útil para **depurar problemas de compilação**: em qual fase falhou?
- Projetos grandes compilam cada .c em .o separadamente — é assim que o **Make** vai acelerar builds



Notes

Warnings: o compilador tentando te ajudar

```
1 $ gcc -Wall -Wextra -std=c11 -g programa.c -o programa
```

- `-Wall` — avisos "essenciais" (variável não usada, comparação suspeita...)
- `-Wextra` — avisos adicionais, mais rigorosos
- `-Wpedantic` — exige aderência estrita ao padrão da linguagem
- `-Werror` — transforma **todo warning em erro** (não compila!)

Boa prática profissional

Compile sempre com `-Wall -Wextra` ligados. Em CI/CD, é comum usar `-Werror` para impedir que warnings cheguem à produção.



Notes

Por que isso importa: um exemplo real

```
1 int fator_bonus = 2; // declarada e nunca usada
2 ...
3 float media = total / 5; // divisao inteira: bug silencioso!
```

```
1 $ gcc -Wall warnings.c -o warnings
2 warnings.c:24:9: warning: unused variable 'fator_bonus'
3             [-Wunused-variable]
```

O warning pega um bug, mas não todos

`-Wall` avisa sobre a variável não usada — mas **não** avisa sobre a divisão inteira, que também é um bug! Warnings ajudam, mas não substituem teste e depuração (chegaremos no GDB já já).



Notes

```
1 $ gcc -O0 programa.c -o programa # sem otimizacao (padrao)
2 $ gcc -O1 programa.c -o programa # otimizacoes basicas
3 $ gcc -O2 programa.c -o programa # recomendado para producao
4 $ gcc -O3 programa.c -o programa # agressivo (vetorizacao, inline)
5 $ gcc -Os programa.c -o programa # otimiza para tamanho
6 $ gcc -Og programa.c -o programa # otimiza mantendo debug facil
```

Durante desenvolvimento

- -O0 ou -Og + -g
- Compila rápido, depura fácil

Para entregar/medir

- -O2 (padrão de mercado)
- Código mais rápido, mais difícil de depurar



Notes

Otimização na prática: medindo o efeito

Multiplicação de matrizes 450×450 – mesmo código-fonte:

```
1 $ gcc -O0 otimizacao.c -o otimizacao_00
2 $ gcc -O2 otimizacao.c -o otimizacao_02
3
4 $ time ./otimizacao_00
5 real 0m0.378s
6
7 $ time ./otimizacao_02
8 real 0m0.071s
```

$\approx 5\times$ mais rápido

Mesmo código, mesma saída (checksum idêntico) – só a flag de otimização mudou. Vocês vão reproduzir essa medição na atividade prática.



Notes

Receita completa recomendada

```
1 $ gcc -Wall -Wextra -g -std=c11 -O0 programa.c -o programa
```

- -Wall -Wextra — avisos ligados
- -g — inclui símbolos de depuração (essencial para o GDB!)
- -std=c11 — fixa o padrão da linguagem, evita surpresas
- -O0 — sem otimização, enquanto o código ainda tem bugs



Notes

Make: automatizando o build

Notes

O problema

Projeto com 10 arquivos .c. Você mudou **um** deles. Vai digitar o comando `gcc` inteiro de novo, recompilando tudo?

Make resolve isso:

- Recompila **só o que mudou** (compara datas de modificação)
- Documenta como o projeto é construído (é a “receita” do build)
- Padrão de fato em projetos C/C++ no Linux há décadas



Anatomia de uma regra

```
1 alvo: pre-requisitos
2 <TAB>comando
```

- **Alvo** (*target*): o que se quer gerar (ex.: um .o, um executável)
- **Pré-requisitos**: arquivos dos quais o alvo depende
- **Comando**: como gerar o alvo — **precisa começar com TAB**, não espaços!

```
1 turma: main.o estatistica.o
2 gcc -o turma main.o estatistica.o
```

Erro clássico

Makefile:3: *** missing separator. Stop. → você usou espaços em vez de TAB.



Variáveis no Makefile

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -g -std=c11
3 TARGET = turma
4 OBJS = main.o estatistica.o
```

Variáveis automáticas (dentro de uma regra):

- `$@` — o nome do alvo
- `$<` — o primeiro pré-requisito
- `$^` — **todos** os pré-requisitos

```
1 $(TARGET): $(OBJS)
2     $(CC) $(CFLAGS) -o $@ $^
```



Notes

Regras padrão e .PHONY

```
1 %.o: %.c
2     $(CC) $(CFLAGS) -c $< -o $@
```

Regra de padrão (*pattern rule*): compila **qualquer** `.c` em `.o`, sem repetir para cada arquivo.

```
1 .PHONY: all clean run
2
3 clean:
4     rm -f $(TARGET) $(OBJS)
```

- `.PHONY` avisa o Make que `clean/run` **não são arquivos** — sempre executa, mesmo se existir um arquivo chamado `clean`



Notes

Makefile completo (testado)

```
1 CC = gcc
2 CFLAGS = -Wall -Wextra -g -std=c11
3 TARGET = turma
4 OBJS = main.o estatistica.o
5 .PHONY: all clean run
6 all: $(TARGET)
7 $(TARGET): $(OBJS)
8     $(CC) $(CFLAGS) -o $@ $^
9 %.o: %.c
10     $(CC) $(CFLAGS) -c $< -o $@
11 run: $(TARGET)
12     ./$$(TARGET)
13 clean:
14     rm -f $(TARGET) $(OBJS)
```



Notes

Make em ação

```
1 $ make
2 gcc -Wall -Wextra -g -std=c11 -c main.c -o main.o
3 gcc -Wall -Wextra -g -std=c11 -c estatistica.c -o estatistica.o
4 gcc -Wall -Wextra -g -std=c11 -o turma main.o estatistica.o
5
6 $ make          # nada mudou
7 make: Nothing to be done for 'all'.
8
9 $ touch main.c  # simula uma edicao
10 $ make          # so recompila o que precisa!
11 gcc -Wall -Wextra -g -std=c11 -c main.c -o main.o
12 gcc -Wall -Wextra -g -std=c11 -o turma main.o estatistica.o
```

estatistica.c **não** foi recompilado — ele não mudou.



Notes

GDB: depurando programas

Notes

Por que um depurador?

Jeito ingênuo

- Espalhar `printf` pelo código
- Recompilar e rodar de novo
- Repetir até achar o bug
- Remover os `printf` depois (ou esquecer!)

Com GDB

- Pausa o programa **em qualquer linha**
- Inspecciona variáveis **sem recompilar**
- Executa **um passo por vez**
- Vê a pilha de chamadas inteira

Notes



```
1 $ gcc -g -Wall -Wextra -std=c11 -o notas notas.c
2 $ gdb ./notas
3 (gdb) break calcula_media
4 (gdb) run
5 (gdb) next
6 (gdb) print soma
7 (gdb) continue
8 (gdb) quit
```

Pré-requisito

Compile sempre com `-g`. Sem símbolos de depuração, o GDB não consegue mostrar nomes de variáveis nem linhas do código-fonte.



Notes

Comandos essenciais do GDB

Comando	O que faz
<code>break <local></code>	Define um ponto de parada (breakpoint)
<code>run</code>	Inicia a execução
<code>next (n)</code>	Executa a linha atual (não entra em funções)
<code>step (s)</code>	Executa a linha atual (entra em funções)
<code>continue (c)</code>	Continua até o próximo breakpoint
<code>print <expr> (p)</code>	Mostra o valor de uma variável/expressão
<code>list (l)</code>	Mostra o código-fonte ao redor da linha atual



Notes

```
watch <variável>
```

Cria um *watchpoint*: o GDB pausa automaticamente **sempre que o valor da variável mudar**. Ótimo para achar *onde* um valor é corrompido.

```
backtrace (bt)
```

Mostra a **pilha de chamadas**: qual função chamou qual, até chegar onde você está agora. Essencial quando o bug aparece longe de onde a função foi chamada.



Sessão de depuração: o bug em notas.c

```
1 double calcula_media(int *notas, int n) {
2     int soma = 0;
3     int i;
4     for (i = 0; i <= n; i++) { // BUG!
5         soma = soma + notas[i];
6     }
7     return (double) soma / n;
8 }
```

```
1 $ ./notas
2 Media da turma: 6561.00    # deveria ser 8.00 !!
```

BUG: a condição deveria ser `i < n`; com `i <= n` o laço lê uma posição além do fim do vetor.

Vamos usar `break`, `watch i` e `backtrace` para encontrar exatamente onde isso acontece no laboratório.



Encerramento

Notes

Síntese: as três ferramentas

Notes

Ferramenta	Pergunta que responde	Quando usar
GCC	Como vira executável?	Sempre
Make	Como automatizar o build?	Projeto com > 1 arquivo
GDB	Por que está errado?	Quando o resultado não bate

Importante dominar essas ferramentas para um bom desenvolvimento em sistemas Linux.



Atividade prática de hoje

No laboratório, vocês vão:

- 1 Compilar um programa com diferentes flags e comparar avisos e tempos
- 2 Escrever um **Makefile do zero** para um projeto com múltiplos arquivos
- 3 Depurar um programa com **bug intencional** usando GDB (break, watch, backtrace)

Guia da atividade

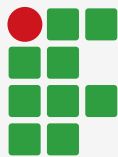
Todas as instruções, código-fonte e critérios de avaliação estão no **guia de atividade prática** do Laboratório 01.

Bibliografia de apoio

- Manuais: `man gcc`, `man make`, `man gdb`
- GNU Make Manual — <https://www.gnu.org/software/make/manual/>
- GDB User Manual — <https://sourceware.org/gdb/documentation/>



That's all folks!



**INSTITUTO
FEDERAL**
Santa Catarina
Câmpus
Florianópolis



Notes

Notes
